

A. THEORY

#1 Why opcode + function fields instead of only opcode? 22-1a

Official key: with only opcode the decoder complexity would increase from 62² to 12²·12

A 6-bit opcode can encode only 2⁶=64 instructions. All R-type instrs share opcode 000000 and are told apart by the 6-bit function → 64 more ALU ops without widening the 32-bit ALU result.
R-type needs no 16-bit immediate, so its low bits are free for shamt-funct: I/1) types need both for imm/address, so they get unique opcodes. Keeps all instrs 32-bit with fixed positions (regularity → simple hardware).

Decoding begins with two levels: main decoder looks only at opcode (6 inputs); ALU decoder uses funct+ALUOP → smaller, faster control (see #10).

#2 Why is single-cycle control called a "Truth Table"? 21-2b, 22-2b

Official key: "No next state." In single-cycle MIPS every instr finishes in one clock, so the control unit is **purely combinational**: no state/memory, its outputs (RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemRead, ALUOp, Jump) depend only on the present inputs (opcode, funct). A combinational circuit is completely specified by listing outputs for every input combination = a truth table (one row per opcode), which is then implemented as SOP logic/PLA/ROM. (Multicycle control is an FSM → state table, not truth table.)

#3 How does the processor capture the side effects when instrs complete? 21-1c

Side effects = changes to computer state: **register file, data memory and PC**. Combinational logic computes the new values during the cycle; they are captured only at the **rising clock edge** by state elements whose write enable is asserted. RF: written if RegWrite=1 (WD3-reg A3), memory written if MemWrite=1 (WD+M[A]), PC-PC every cycle (Multicycle: PCEn). Intermediate results at memory/pipeline are held in non-architectural registers (R, MDR, A, ALUOP / pipeline regs) and committed in the write-back stage.

#4 <2 behaves as <1 → which instrs affected? 21-1d, 22-1d, 23-2b

Shift-by-2 converts a word offset to a byte offset; used only for PC-relative/jump targets. **Always state the effect** (graded 2025 lost marks for only saying "last 2 bits must be 00").

Official key: BNE, J → instruction alignment problem.
bne (branch): PCBranch = PC + 4 (Sigmoid) → PC+4+imm instead of 4+imm → wrong, half-word aligned address (e.g., imm=3 → PC+4+6 not PC+4+12) → fetches garbage/misaligned instr. Not-taken branch stall (OK (PC+4)).

J (and jal): JTA = [PC+31:28], add<rc-1> <26b-1> 2-bits only, target = half the intended address, wrong/misaligned → jumps to wrong location. Same in multicycle (ALUOp+PC+Sigmoid<imm> in 2 Decode; jump path). **Not affected:** R-type type, sub, and, or, lwi, sw, addi → they don't use the shifters (lwi/sw offsets are already byte offsets).

#5 Stuck control signals (single-cycle) 21-1a, 23-1c

Signal	Malfunction & why
MemtoReg	R-type (add,sub,and,or,sl): addi write ReadData (=M[ALUSResult]) to RegA1; reg instead of ALUSResult, OK (OK (RegWrite=0))
MemtoReg	lw only: writes ALUSResult (the address RF[r3]+imm) into t instead of load register. All others OK.
ALUOp	ALUOp=00→01 → ALU subtracts: lw, sw (address = rs+imm), addi (rs+imm) wrong, beg (01) OK. R-type: 10→11 → ALU decoder locks at funct when ALUOp=1 (11 treated as 10) → OK (if 11 were undefined, R-type would also fail). J OK.
RegDst=0	R-type: result written to R1 (20:16) instead of rd (15:11) → wrong reg & corrupts r1. lw/addi OK (use rt: sw, beq, lrd (no write)).

#6 Can it be the best for R-type (and for R-type)? 23-2c

Not advisable - no, MIPS reads registers from fixed fields: Read reg1 = rs (25:21), Read reg2 = rt (20:16) for all instrs, so the RF can be read in **parallel with decoding**, sw and beq must read r1 as a source; if R-type wrote rt, its 2nd source would be in rd, so a mux (controlled by decoder) would be needed on **Read register 2** → RF read starts only after control decode → longer critical path (+1st MUX). Vice-versa (if write wiring rd) is impossible: bits 15:11 are part of the 16-bit immediate in lw/addi. Hence dest mux (RegDst) is on the write side: r1 for l-type, rd for R-type.

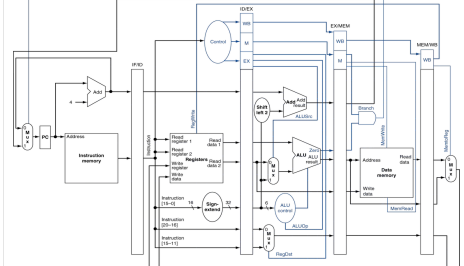
#7 Do we need opcode in a special-purpose processor? 23-1c

Not advisable - no, MIPS reads registers from fixed fields: Read reg1 = rs (25:21), Read reg2 = rt (20:16) for all instrs, so the RF can be read in **parallel with decoding**, sw and beq must read r1 as a source; if R-type wrote rt, its 2nd source would be in rd, so a mux (controlled by decoder) would be needed on **Read register 2** → RF read starts only after control decode → longer critical path (+1st MUX). Vice-versa (if write wiring rd) is impossible: bits 15:11 are part of the 16-bit immediate in lw/addi. Hence dest mux (RegDst) is on the write side: r1 for l-type, rd for R-type.

#8 Single-cycle vs multi-cycle control unit 24-1a

Single-cycle	Multi-cycle
Combinational (truth table): signals = {opcode,Funct}	Sequential FSM: state reg + next-state logic + outputs; signals = {fstate}, Moore, next state = {fstate}, outputs = {fdata}
Signals constant for whole (long) cycle: PC=1	Signals change every state: PC=1-5

Signals: RegDst, RegWrite, ALUSrc, Branch, MemWrite, MemtoReg, ALUOp, Jump
Separate I/O memories & address sharing: MDR & ALU reused across states → no sequencing
Only hardware: Hardwired FSM (Multicycle programmed) (P9)
Control table: Jump, RegDst, ALUSrc, Branch, ALUOp, MemRead, MemWrite, MemtoReg, ALUOp, Jump
TO-T1T2-W1W2-W3-W4-W5-W6-W7-W8-W9-W10-W11-W12-W13-W14-W15-W16-W17-W18-W19-W20-W21-W22-W23-W24-W25-W26-W27-W28-W29-W30-W31-W32-W33-W34-W35-W36-W37-W38-W39-W40-W41-W42-W43-W44-W45-W46-W47-W48-W49-W50-W51-W52-W53-W54-W55-W56-W57-W58-W59-W60-W61-W62-W63-W64-W65-W66-W67-W68-W69-W70-W71-W72-W73-W74-W75-W76-W77-W78-W79-W80-W81-W82-W83-W84-W85-W86-W87-W88-W89-W90-W91-W92-W93-W94-W95-W96-W97-W98-W99-W100-W101-W102-W103-W104-W105-W106-W107-W108-W109-W110-W111-W112-W113-W114-W115-W116-W117-W118-W119-W120-W121-W122-W123-W124-W125-W126-W127-W128-W129-W130-W131-W132-W133-W134-W135-W136-W137-W138-W139-W140-W141-W142-W143-W144-W145-W146-W147-W148-W149-W150-W151-W152-W153-W154-W155-W156-W157-W158-W159-W160-W161-W162-W163-W164-W165-W166-W167-W168-W169-W170-W171-W172-W173-W174-W175-W176-W177-W178-W179-W180-W181-W182-W183-W184-W185-W186-W187-W188-W189-W190-W191-W192-W193-W194-W195-W196-W197-W198-W199-W200-W201-W202-W203-W204-W205-W206-W207-W208-W209-W210-W211-W212-W213-W214-W215-W216-W217-W218-W219-W220-W221-W222-W223-W224-W225-W226-W227-W228-W229-W230-W231-W232-W233-W234-W235-W236-W237-W238-W239-W240-W241-W242-W243-W244-W245-W246-W247-W248-W249-W250-W251-W252-W253-W254-W255-W256-W257-W258-W259-W260-W261-W262-W263-W264-W265-W266-W267-W268-W269-W270-W271-W272-W273-W274-W275-W276-W277-W278-W279-W280-W281-W282-W283-W284-W285-W286-W287-W288-W289-W290-W291-W292-W293-W294-W295-W296-W297-W298-W299-W300-W301-W302-W303-W304-W305-W306-W307-W308-W309-W310-W311-W312-W313-W314-W315-W316-W317-W318-W319-W320-W321-W322-W323-W324-W325-W326-W327-W328-W329-W330-W331-W332-W333-W334-W335-W336-W337-W338-W339-W340-W341-W342-W343-W344-W345-W346-W347-W348-W349-W350-W351-W352-W353-W354-W355-W356-W357-W358-W359-W360-W361-W362-W363-W364-W365-W366-W367-W368-W369-W370-W371-W372-W373-W374-W375-W376-W377-W378-W379-W380-W381-W382-W383-W384-W385-W386-W387-W388-W389-W390-W391-W392-W393-W394-W395-W396-W397-W398-W399-W400-W401-W402-W403-W404-W405-W406-W407-W408-W409-W410-W411-W412-W413-W414-W415-W416-W417-W418-W419-W420-W421-W422-W423-W424-W425-W426-W427-W428-W429-W430-W431-W432-W433-W434-W435-W436-W437-W438-W439-W440-W441-W442-W443-W444-W445-W446-W447-W448-W449-W450-W451-W452-W453-W454-W455-W456-W457-W458-W459-W460-W461-W462-W463-W464-W465-W466-W467-W468-W469-W470-W471-W472-W473-W474-W475-W476-W477-W478-W479-W480-W481-W482-W483-W484-W485-W486-W487-W488-W489-W490-W491-W492-W493-W494-W495-W496-W497-W498-W499-W500-W501-W502-W503-W504-W505-W506-W507-W508-W509-W510-W511-W512-W513-W514-W515-W516-W517-W518-W519-W520-W521-W522-W523-W524-W525-W526-W527-W528-W529-W530-W531-W532-W533-W534-W535-W536-W537-W538-W539-W540-W541-W542-W543-W544-W545-W546-W547-W548-W549-W550-W551-W552-W553-W554-W555-W556-W557-W558-W559-W560-W561-W562-W563-W564-W565-W566-W567-W568-W569-W570-W571-W572-W573-W574-W575-W576-W577-W578-W579-W580-W581-W582-W583-W584-W585-W586-W587-W588-W589-W590-W591-W592-W593-W594-W595-W596-W597-W598-W599-W600-W601-W602-W603-W604-W605-W606-W607-W608-W609-W610-W611-W612-W613-W614-W615-W616-W617-W618-W619-W620-W621-W622-W623-W624-W625-W626-W627-W628-W629-W630-W631-W632-W633-W634-W635-W636-W637-W638-W639-W640-W641-W642-W643-W644-W645-W646-W647-W648-W649-W650-W651-W652-W653-W654-W655-W656-W657-W658-W659-W660-W661-W662-W663-W664-W665-W666-W667-W668-W669-W670-W671-W672-W673-W674-W675-W676-W677-W678-W679-W680-W681-W682-W683-W684-W685-W686-W687-W688-W689-W690-W691-W692-W693-W694-W695-W696-W697-W698-W699-W700-W701-W702-W703-W704-W705-W706-W707-W708-W709-W710-W711-W712-W713-W714-W715-W716-W717-W718-W719-W720-W721-W722-W723-W724-W725-W726-W727-W728-W729-W730-W731-W732-W733-W734-W735-W736-W737-W738-W739-W740-W741-W742-W743-W744-W745-W746-W747-W748-W749-W750-W751-W752-W753-W754-W755-W756-W757-W758-W759-W760-W761-W762-W763-W764-W765-W766-W767-W768-W769-W770-W771-W772-W773-W774-W775-W776-W777-W778-W779-W780-W781-W782-W783-W784-W785-W786-W787-W788-W789-W790-W791-W792-W793-W794-W795-W796-W797-W798-W799-W800-W801-W802-W803-W804-W805-W806-W807-W808-W809-W810-W811-W812-W813-W814-W815-W816-W817-W818-W819-W820-W821-W822-W823-W824-W825-W826-W827-W828-W829-W830-W831-W832-W833-W834-W835-W836-W837-W838-W839-W840-W841-W842-W843-W844-W845-W846-W847-W848-W849-W850-W851-W852-W853-W854-W855-W856-W857-W858-W859-W860-W861-W862-W863-W864-W865-W866-W867-W868-W869-W870-W871-W872-W873-W874-W875-W876-W877-W878-W879-W880-W881-W882-W883-W884-W885-W886-W887-W888-W889-W890-W891-W892-W893-W894-W895-W896-W897-W898-W899-W900-W901-W902-W903-W904-W905-W906-W907-W908-W909-W910-W911-W912-W913-W914-W915-W916-W917-W918-W919-W920-W921-W922-W923-W924-W925-W926-W927-W928-W929-W930-W931-W932-W933-W934-W935-W936-W937-W938-W939-W940-W941-W942-W943-W944-W945-W946-W947-W948-W949-W950-W951-W952-W953-W954-W955-W956-W957-W958-W959-W960-W961-W962-W963-W964-W965-W966-W967-W968-W969-W970-W971-W972-W973-W974-W975-W976-W977-W978-W979-W980-W981-W982-W983-W984-W985-W986-W987-W988-W989-W990-W991-W992-W993-W994-W995-W996-W997-W998-W999-W1000-W1001-W1002-W1003-W1004-W1005-W1006-W1007-W1008-W1009-W1010-W1011-W1012-W1013-W1014-W1015-W1016-W1017-W1018-W1019-W1020-W1021-W1022-W1023-W1024-W1025-W1026-W1027-W1028-W1029-W1030-W1031-W1032-W1033-W1034-W1035-W1036-W1037-W1038-W1039-W1040-W1041-W1042-W1043-W1044-W1045-W1046-W1047-W1048-W1049-W1050-W1051-W1052-W1053-W1054-W1055-W1056-W1057-W1058-W1059-W1060-W1061-W1062-W1063-W1064-W1065-W1066-W1067-W1068-W1069-W1070-W1071-W1072-W1073-W1074-W1075-W1076-W1077-W1078-W1079-W1080-W1081-W1082-W1083-W1084-W1085-W1086-W1087-W1088-W1089-W1090-W1091-W1092-W1093-W1094-W1095-W1096-W1097-W1098-W1099-W1100-W1101-W1102-W1103-W1104-W1105-W1106-W1107-W1108-W1109-W1110-W1111-W1112-W1113-W1114-W1115-W1116-W1117-W1118-W1119-W1120-W1121-W1122-W1123-W1124-W1125-W1126-W1127-W1128-W1129-W1130-W1131-W1132-W1133-W1134-W1135-W1136-W1137-W1138-W1139-W1140-W1141-W1142-W1143-W1144-W1145-W1146-W1147-W1148-W1149-W1150-W1151-W1152-W1153-W1154-W1155-W1156-W1157-W1158-W1159-W1160-W1161-W1162-W1163-W1164-W1165-W1166-W1167-W1168-W1169-W1170-W1171-W1172-W1173-W1174-W1175-W1176-W1177-W1178-W1179-W1180-W1181-W1182-W1183-W1184-W1185-W1186-W1187-W1188-W1189-W1190-W1191-W1192-W1193-W1194-W1195-W1196-W1197-W1198-W1199-W1200-W1201-W1202-W1203-W1204-W1205-W1206-W1207-W1208-W1209-W1210-W1211-W1212-W1213-W1214-W1215-W1216-W1217-W1218-W1219-W1220-W1221-W1222-W1223-W1224-W1225-W1226-W1227-W1228-W1229-W1230-W1231-W1232-W1233-W1234-W1235-W1236-W1237-W1238-W1239-W1240-W1241-W1242-W1243-W1244-W1245-W1246-W1247-W1248-W1249-W1250-W1251-W1252-W1253-W1254-W1255-W1256-W1257-W1258-W1259-W1260-W1261-W1262-W1263-W1264-W1265-W1266-W1267-W1268-W1269-W1270-W1271-W1272-W1273-W1274-W1275-W1276-W1277-W1278-W1279-W1280-W1281-W1282-W1283-W1284-W1285-W1286-W1287-W1288-W1289-W1290-W1291-W1292-W1293-W1294-W1295-W1296-W1297-W1298-W1299-W1300-W1301-W1302-W1303-W1304-W1305-W1306-W1307-W1308-W1309-W1310-W1311-W1312-W1313-W1314-W1315-W1316-W1317-W1318-W1319-W1320-W1321-W1322-W1323-W1324-W1325-W1326-W1327-W1328-W1329-W1330-W1331-W1332-W1333-W1334-W1335-W1336-W1337-W1338-W1339-W1340-W1341-W1342-W1343-W1344-W1345-W1346-W1347-W1348-W1349-W1350-W1351-W1352-W1353-W1354-W1355-W1356-W1357-W1358-W1359-W1360-W1361-W1362-W1363-W1364-W1365-W1366-W1367-W1368-W1369-W1370-W1371-W1372-W1373-W1374-W1375-W1376-W1377-W1378-W1379-W1380-W1381-W1382-W1383-W1384-W1385-W1386-W1387-W1388-W1389-W1390-W1391-W1392-W1393-W1394-W1395-W1396-W1397-W1398-W1399-W1400-W1401-W1402-W1403-W1404-W1405-W1406-W1407-W1408-W1409-W1410-W1411-W1412-W1413-W1414-W1415-W1416-W1417-W1418-W1419-W1420-W1421-W1422-W1423-W1424-W1425-W1426-W1427-W1428-W1429-W1430-W1431-W1432-W1433-W1434-W1435-W1436-W1437-W1438-W1439-W1440-W1441-W1442-W1443-W1444-W1445-W1446-W1447-W1448-W1449-W1450-W1451-W1452-W1453-W1454-W1455-W1456-W1457-W1458-W1459-W1460-W1461-W1462-W1463-W1464-W1465-W1466-W1467-W1468-W1469-W1470-W1471-W1472-W1473-W1474-W1475-W1476-W1477-W1478-W1479-W1480-W1481-W1482-W1483-W1484-W1485-W1486-W1487-W1488-W1489-W1490-W1491-W1492-W1493-W1494-W1495-W1496-W1497-W1498-W1499-W1500-W1501-W1502-W1503-W1504-W1505-W1506-W1507-W1508-W1509-W1510-W1511-W1512-W1513-W1514-W1515-W1516-W1517-W1518-W1519-W1520-W1521-W1522-W1523-W1524-W1525-W1526-W1527-W1528-W1529-W1530-W1531-W1532-W1533-W1534-W1535-W1536-W1537-W1538-W1539-W1540-W1541-W1542-W1543-W1544-W1545-W1546-W1547-W1548-W1549-W1550-W1551-W1552-W1553-W1554-W1555-W1556-W1557-W1558-W1559-W1560-W1561-W1562-W1563-W1564-W1565-W1566-W1567-W1568-W1569-W1570-W1571-W1572-W1573-W1574-W1575-W1576-W1577-W1578-W1579-W1580-W1581-W1582-W1583-W1584-W1585-W1586-W1587-W1588-W1589-W1590-W1591-W1592-W1593-W1594-W1595-W1596-W1597-W1598-W1599-W1600-W1601-W1602-W1603-W1604-W1605-W1606-W1607-W1608-W1609-W1610-W1611-W1612-W1613-W1614-W1615-W1616-W1617-W1618-W1619-W1620-W1621-W1622-W1623-W1624-W1625-W1626-W1627-W1628-W1629-W1630-W1631-W1632-W1633-W1634-W1635-W1636-W1637-W1638-W1639-W1640-W1641-W1642-W1643-W1644-W1645-W1646-W1647-W1648-W1649-W1650-W1651-W1652-W1653-W1654-W1655-W1656-W1657-W1658-W1659-W1660-W1661-W1662-W1663-W1664-W1665-W1666-W1667-W1668-W1669-W1670-W1671-W1672-W1673-W1674-W1675-W1676-W1677-W1678-W1679-W1680-W1681-W1682-W1683-W1684-W1685-W1686-W1687-W1688-W1689-W1690-W1691-W1692-W1693-W1694-W1695-W1696-W1697-W1698-W1699-W1700-W1701-W1702-W1703-W1704-W1705-W1706-W1707-W1708-W1709-W1710-W1711-W1712-W1713-W1714-W1715-W1716-W1717-W1718-W1719-W1720-W1721-W1722-W1723-W1724-W1725-W1726-W1727-W1728-W1729-W1730-W1731-W1732-W1733-W1734-W1735-W1736-W1737-W1738-W1739-W1740-W1741-W1742-W1743-W1744-W1745-W1746-W1747-W1748-W1749-W1750-W1751-W1752-W1753-W1754-W1755-W1756-W1757-W1758-W1759-W1760-W1761-W1762-W1763-W1764-W1765-W1766-W1767-W1768-W1769-W1770-W1771-W1772-W1773-W1774-W1775-W1776-W1777-W1778-W1779-W1780-W1781-W1782-W1783-W1784-W1785-W1786-W1787-W1788-W1789-W1790-W1791-W1792-W1793-W1794-W1795-W1796-W1797-W1798-W1799-W1800-W1801-W1802-W1803-W1804-W1805-W1806-W1807-W1808-W1809-W1810-W1811-W1812-W1813-W1814-W1815-W1816-W1817-W1818-W1819-W1820-W1821-W1822-W1823-W1824-W1825-W1826-W1827-W1828-W1829-W1830-W1831-W1832-W1833-W1834-W1835-W1836-W1837-W1838-W1839-W1840-W1841-W1842-W1843-W1844-W1845-W1846-W1847-W1848-W1849-W1850-W1851-W1852-W1853-W1854-W1855-W1856-W1857-W1858-W1859-W1860-W1861-W1862-W1863-W1864-W1865-W1866-W1867-W1868-W1869-W1870-W1871-W1872-W1873-W1874-W1875-W1876-W1877-W1878-W1879-W1880-W1881-W1882-W1883-W1884-W1885-W1886-W1887-W1888-W1889-W1890-W1891-W1892-W1893-W1894-W1895-W1896-W1897-W1898-W1899-W1900-W1901-W1902-W1903-W1904-W1905-W1906-W1907-W1908-W1909-W1910-W1911-W1912-W1913-W1914-W1915-W1916-W1917-W1918-W1919-W1920-W1921-W1922-W1923-W1924-W1925-W1926-W1927-W1928-W1929-W1930-W1931-W1932-W1933-W1934-W1935-W1936-W1937-W1938-W1939-W1940-W1941-W1942-W1943-W1944-W1945-W1946-W1947-W1948-W1949-W1950-W1951-W1952-W1953-W1954-W1955-W1956-W1957-W1958-W1959-W1960-W1961-W1962-W1963-W1964-W1965-W1966-W1967-W1968-W1969-W1970-W1971-W1972-W1973-W1974-W1975-W1976-W1977-W1978-W1979-W1980-W1981-W1982-W1983-W1984-W1985-W1986-W1987-W1988-W1989-W1990-W1991-W1992-W1993-W1994-W1995-W1996-W1997-W1998-W1999-W2000-W2001-W2002-W2003-W2004-W2005-W2006-W2007-W2008-W2009-W2010-W2011-W2012-W2013-W2014-W2015-W2016-W2017-W2018-W2019-W2020-W2021-W2022-W2023-W2024-W2025-W2026-W2027-W2028-W2029-W2030-W2031-W2032-W2033-W2034-W2035-W2036-W2037-W2038-W2039-W2040-W2041-W2042-W2043-W2044-W2045-W2046-W2047-W2048-W2049-W2050-W2051-W2052-W2053-W2054-W2055-W2056-W2057-W2058-W2059-W2060-W2061-W2062-W2063-W2064-W2065-W2066-W2067-W2068-W2069-W2070-W2071-W2072-W2073-W2074-W2075-W2076-W2077-W2078-W2079-W2080-W2081-W2082-W2083-W2084-W2085-W2086-W2087-W2088-W2089-W2090-W2091-W2092-W2093-W2094-W2095-W2096-W2097-W2098-W2099-W2100-W2101-W2102-W2103-W2104-W2105-W2106-W2107-W2108-W2109-W2110-W2111-W2112-W2113-W2114-W2115-W2116-W2117-W2118-W2119-W2120-W2121-W2122-W2123-W2124-W2125-W2126-W2127-W2128-W2129-W2130-W2131-W2132-W2133-W2134-W2135-W2136-W2137-W2138-W2139-W2140-W2141-W2142-W2143-W2144-W2145-W2146-W2147-W2148-W2149-W2150-W2151-W2152-W2153-W2154-W2155-W2156-W2157-W2158-W2159-W2160-W2161-W2162-W2163-W2164-W2165-W2166-W2167-W2168-W2169-W2170-W2171-W2172-W2173-W2174-W2175-W2176-W2177-W2178-W2179-W2180-W2181-W2182-W2183-W2184-W2185-W2186-W2187



Pipelined datapath + control (slide 32, P&H): control in ID → WB/M/EX fields in ID/EX (9b), EX/MEM (5b), MEM/WB (2b).

```

#define OPF0D0E 0xFC000000 // 0b111111 then 26 zeros
#define RS      0x03C00000 // bits 25:21
#define RT      0x00B00000 // bits 20:16
#define RD      0x00A00000 // bits 19:15
#define SHIFT   0x000007C0 // bits 10:6
#define OFFSET  0x00000FFF // bits 15:0

int PC, IMM[1024], MM[1024], RF[32], ALUControl;
bool ZERO;
int SE(int x){ return (short)(x); } // 16-32 bits
int ALU(int i1, int s1, s2){ switch(ALUControl){
    case 0b0110: return i1+s1-s2; return s1-s2; } // sub/R
    case 0b010:  return s1&s2;      case 0b0001: return s1&s2;
    case 0b0000: return s1&s2;      case 0b0010: return s1|s2;
    case 0b0111: return s1<s2; }
load[IMM, DMM]; PC = address of 1st instruction;
while(1){ int i = IMM[PC];

```

```

switch((I & 0b0000) == 26;
case C.TYPE: ALUControl = 0; from funct; // ADD, SUB, AND, OR
RF[IA&0] == 11; // ALU(RF[IA&8] == 21, RF[IA&T] == 16]);
PC = PC + 4; break;
case S: ALUControl = 0b0010;
DMW[ALU(RF[IA&8] == 21, SE[IA&OFFSET])] = RF[IA&T] == 16;
PC = PC + 4; break;
case S1: ALUControl = 0b0010;
RF[IA&T] == 16; // DMW[ALU(RF[IA&8] == 21, SE[IA&OFFSET])];
PC = PC + 4; break;
case B: ALUControl = 0b0110;
ALU(RF[IA&8] == 21, RF[IA&T] == 16);
PC = PC + 4; break;
case J: ALUControl = 0b0000; PC = PC + 4; PC += SE[IA&OFFSET] << 2;

```

```

    else PC = PC + 4; break;
    case ADDI: ALUControl = 0b010; // homework
    RF[(&RT)>>16] = ALU(RF[(&RS)>>21], SE(&OFSFSET));
    case JALR: PC = PC + 4; break;
    case J: PC = ((PC*4) & 0xF0000000) | ((I & 0x3FFFFFFF) <<
2);
}
}

```

R14 Prof's C++ -- multi-cycle (M1-d slide 15): one memory MM, ALU reused
 int PC, MM[2042], RF[32], ALUControl; bool ZERO;
 int R, A, B, ALUOUT, MDR; // non-architectural
 load(MM, PC, 1st, 1st);

```
IR = MM[PC]; ALUControl = @0b010; PC = ALU(PC, 4); //T0
A = RF[IR&R8]>>21; B = RF[IR&R7]>>16]; //T1
switch(IR & OPCODE) >> 26 {
case R_TYPE: ALUControl = from funcnt;
    ALUOut = ALU(A, B); // T6 (T2)
    RF[IR&R8]>>11 = ALUOut; break; // T7 (T3)
case LW: ALUControl = @0b010;
    ALUOut = ALU(A, SE(IR&OFFSET)); // T2
    MDR = MM[ALUOut]; // T3
```

```

    RF[(&TOUT)>=16] = MDR; break; // T4
case SW: ALUControl = @0b0110; // T5
        ALUOut = ALU(A, SE(IR[OFFSET])); // T2
        MM[ALUOut] = B; break; // T5 (T3)
case B_TYPE: ALUControl = @0b0110;
        ALUOut = ALU(PC, SE(IR[OFFSET]<<2)); // T1
        ALUControl = @0b1110; ALUOut = ALU(A, B); // T8
        if (ZERO == 1) PC = ALUOut; break;
case ADDI: ALUControl = @0b0110;
        ALUOut = ALU(A, SE(IR[OFFSET])); // T2
        RF[(&TOUT)>=16] = ALUOut; break; // T9 (T3)
case J: PC = (PC & 0xF0000000) | ((IR[0&x93FFFFF]<<2);

```

```

} } // Cycles: 1W 3, SW 4, R 4, add1 4, beq 3, j 3
} }

R15 Multicyle datapath in Verilog (Harris)
module datapath(input clk, reset, PCen, IorD,
    IRWrite, RegDst, MementoReg, RegWrite, ALUSrcA,
    input [1:0] ALUSrcB, PCSrc, input [2:0] ALUCtrl,
    input [31:0] ReadData, output [5:0] Op, Funct,
    output Zero, output [31:0]Adr, WriteData);

```

```

      SrcB, ALUResult, ALUOut, SignImm, WData, WData[4:0] WA3;
      fpopen #32) pcreg[clk, reset, PCEN, PCNext, PC];
      mux2 #32) adrmux(PC, ALUOut, IOrD, Adr);
      fpopen #32) r1[clk, reset, IRWrite, ReadData, Instr];
      fpopr #32) mdr[clk, reset, ReadData, Data];
      mux2 #5) wamux(Instr[28:16], Instr[15:11],
        RegDst, WA3);
      mux2 #32) wdmux(ALUOut, Data, MemtOrW, WD3);
      regfile rf[clk, RegWrite, Instr[25:21], Instr[28:16],
        WA3, WD3, RDr, Rd2];
      signext se[Instr[15:0], SignImm];
      flgnr #32) wacn[clk, rpsel, Rd1, A];

```

```

bit   f_lopr #(32) breg(clk,reset,RD2,WriteData);
mux2  #(32) srcmux(PC,A,ALUSrcA,SrcA);
mux4  #(32) srcbmux(WriteData,32'd4,SignImm,
                  {SignImm[29:0],2'b00},ALUSrcB,SrcB);
alu    alu(SrcA,SrcB,ALUctrl,ALUResult,Zero);
f_lopr #(32) aluoutreg(clk,reset,ALUResult,ALUOut);

```

```
mux3 <= {32} pncmt(ALUResult, ALUOut,
    {pc[31:0], pcNext}); // 32-bit PCNext
assign PnCrnt = PnCrntNext;
assign FcnctIn[F5:0] = FcnctIn[F5:0];
endmodule

// controller: PCen = PWrite |k (Branch & Zero)
// Floaden: always #posedge clk, posedge reset)
if (!reset) cno; else if(en) cno; // (fload) no en
// regfile: write #posedge if we3; rd = (ra+8)?rf(ra:8)
```

Half adder: $S = A \oplus B$, $Cout = AB$. **Full adder:** $S = A \oplus B \oplus Cin$, $Cout = AB +$

High-level construct	Digital construct	Type
Scalar/variable	register or wire	sequential if reg
Array	memory (write: decoder, read: mux)	sequential
Operators + - * /	functional unit	combinational
If-else / switch / loop	control unit	comb./sequential
Bitwise & OR	peripherals	comb.

if(sel) a<=10; else a=5;	multiplexer	comb.
if(sel) a=10; else b=5;	decoder (enables)	comb.
for(i=103;i>0;i--)	counter (load max, decrement, zero → stop); nested loops: 2 counters, stop = AND of zeros	sequential
a<b, a==b	n-bit comparator (A<B, A=B, A<B)	comb.

```

module d_latch(input d, en, rstn, output reg q);
  always @(en or rstn or d)
    if(en)
      if(rstn)
        q<=0;
      else
        q<=d;
endmodule
  
```

```

    if (en) q <= 0; else if (!en) q <= d;
endmodule

module dff(input clk, d, output reg q);
    always @(posedge clk) q <= d;
endmodule

```

Abstraction layers: Problem → Algorithm → Program/Language → Runtime (OS, VM) → **ISA** → **Microarchitecture** → **Logic** (CS F342 focus) → Devices → Electronics. **ISA** is the computer as seen by the compiler (MIPS, RISC-V, 8085, x86, x64). **MIPS** = Microprocessor without Interlocked Pipelined Stages. **Architecture** = computational structure w.r.t. user: minimal resources, scalable. Two ideas: all computers compute the same things given time &

Moore's law: transistors per chip double every ~2 years (cores after ~2005).
 Computation history: manual → mechanical → electro-mechanical → electrical (vacuum tubes → transistors). FPGA = CLBs of 4-input LUTs.
 Homework: bubble-sort SPP; GP processor with ADD, SUB, LW, SW, BNE, 32 reg. $R0 = 0$

$C =$
 $\Rightarrow$
 a